

RENTHUB

House Rental and Booking Web Application

Project Design & Full Stack Project Documentation

Field	Details
Project Name	RentHub – House Rental and Booking Web Application
Domain	Full Stack Web Development
Frontend	React.js + Vite
Backend	REST API expected at http://localhost:5000/api
Database	MongoDB (recommended/assumed for MERN implementation)
Primary Users	Tenant and Property Owner
Date	14 July 2026

Note: This document is prepared from the submitted RentHub frontend source code and the previously provided project-design templates. Backend implementation details not present in the submitted source are marked as proposed/assumed architecture.

Document Contents

- 1. Introduction and Project Overview
- 2. Empathy Map
- 3. Problem Statements
- 4. Brainstorming and Idea Prioritization
- 5. Problem–Solution Fit
- 6. Proposed Solution
- 7. Solution Requirements: Functional and Non-functional
- 8. Technical Architecture and Technology Stack
- 9. Data Flow and User Stories
- 10. Project Planning and Product Backlog
- 11. User Acceptance Testing
- 12. Full Stack Development Documentation
- 13. Known Issues and Future Enhancements

1. Introduction and Project Overview

1.1 Project Title

RentHub – House Rental and Booking Web Application

1.2 Purpose

RentHub is a web application designed to simplify the process of discovering, listing, and booking rental houses. Tenants can register, log in, search and filter houses, create bookings, and view their bookings. Property owners can register with an owner role, add rental properties, and manage house listings through price updates and deletion.

1.3 Problem Background

Traditional rental-house discovery often depends on brokers, scattered advertisements, social-media posts, or repeated physical visits. Tenants may struggle to compare rental options, while owners may find it difficult to publish and manage property information in one place. RentHub provides a centralized digital workflow for both groups.

1.4 Existing Implemented Frontend Features

- User registration with Tenant or Owner role.
- User login and local storage of authentication token and user information.
- Protected routes for Add House and My Bookings.
- House listing retrieved from the backend API.
- Search houses by location.
- Filter houses by property type and maximum price.
- Create a house booking.
- View the logged-in user's bookings.
- Owner-only user-interface controls for updating house price and deleting a house.
- Add a new house with title, description, location, price, type, bedrooms, bathrooms, image URL, and amenities.
- Responsive styling for home, authentication, and house-listing pages.

2. Empathy Map

User Group	Says	Thinks	Does	Feels
Tenant	"I need a suitable rental house within my budget and preferred location."	"Can I compare options quickly and know the important property details?"	Searches by location, filters by type/price, reviews listings, books a house, checks bookings.	Concerned about time, price, convenience, and availability.
Property Owner	"I want to list my property and reach tenants easily."	"I need a simple way to add and manage my house listing."	Registers as owner, adds property details, updates price, deletes listings.	Wants control, visibility, and a simple management process.

3. Define Problem Statements

PS-1: Tenant

I am a tenant looking for a rental home. I'm trying to find a suitable house based on location, property type, and monthly budget, but rental information can be scattered and difficult to compare because there is no single convenient place to search, filter, and book available houses, which makes me feel frustrated and uncertain.

PS-2: Property Owner

I am a property owner with a house available for rent. I'm trying to reach potential tenants and manage my property listing, but traditional advertising and manual communication can be slow because property information and tenant interactions are not managed through one centralized application, which makes me feel that the rental process is inefficient.

4. Brainstorming and Idea Prioritization

Idea	User Value	Feasibility	Priority	Decision
Centralized house listing	High	High	P0	Implement
Location search	High	High	P0	Implemented
House-type filter	High	High	P0	Implemented
Maximum-price filter	High	High	P0	Implemented
Online booking request	High	High	P0	Implemented
Owner property management	High	High	P0	Implemented in UI
My Bookings dashboard	High	High	P0	Implemented
Image upload from device	Medium	Medium	P1	Future enhancement
Maps and geolocation	High	Medium	P1	Future enhancement
Online rent payment	High	Medium	P1	Future enhancement
Ratings and reviews	Medium	High	P2	Future enhancement
Email/OTP confirmation	Medium	Medium	P2	Future enhancement

5. Problem–Solution Fit

Problem / Need	RentHub Solution	Expected Benefit
Difficulty finding houses in one place	Centralized Available Houses page	Faster discovery
Too many irrelevant listings	Location, type, and price filtering	More relevant results
Manual booking process	Book Now action connected to booking API	Simpler booking request
Owners need listing control	Add House, Update Price, Delete controls	Easier property management
Users need booking visibility	My Bookings page	Centralized booking history/status
Unauthorized access to private pages	ProtectedRoute checks authentication token	Basic client-side route protection

6. Proposed Solution

6.1 Solution Summary

RentHub is a role-based rental property platform that connects tenants and property owners. It provides a React-based user interface and communicates with a REST backend through Axios. The application organizes the rental journey into registration, authentication, property discovery, property management, booking, and booking tracking.

6.2 Uniqueness / Value Proposition

- Single application for both tenants and owners.
- Role-based experience using Tenant and Owner roles.
- Simple house discovery with multiple filters.
- Integrated listing and booking workflow.
- Lightweight, modular React frontend that can be extended with additional services.

6.3 Impact

The solution can reduce the time required to discover rental houses, make property information easier to compare, provide owners with a digital listing channel, and create a structured record of bookings.

6.4 Scalability

The frontend is component-based and communicates through REST APIs, allowing frontend and backend services to evolve independently. A production version can scale through stateless API servers, a managed database, caching, object storage for images, and load balancing.

7. Solution Requirements (Functional & Non-functional)

7.1 Functional Requirements

FR No.	Functional Requirement (Epic)	Sub Requirement / Story
FR-1	User Registration	Register using name, email, password, and role (Tenant/Owner).
FR-2	User Authentication	Login using email and password; receive token and user data.
FR-3	Session Handling	Store token/user locally and allow logout.
FR-4	Access Control	Protect Add House and My Bookings routes for authenticated users.
FR-5	House Discovery	Retrieve and display available houses.
FR-6	House Search	Search houses by location.
FR-7	House Filtering	Filter by house type and maximum monthly price.
FR-8	House Creation	Authenticated user can submit property details to add a house.
FR-9	House Management	Owner UI can update house price and delete a house.
FR-10	Booking	Logged-in user can create a booking for a selected house.
FR-11	Booking History	Logged-in user can view bookings associated with their user ID.
FR-12	Booking Status	Display booking status and booking date.

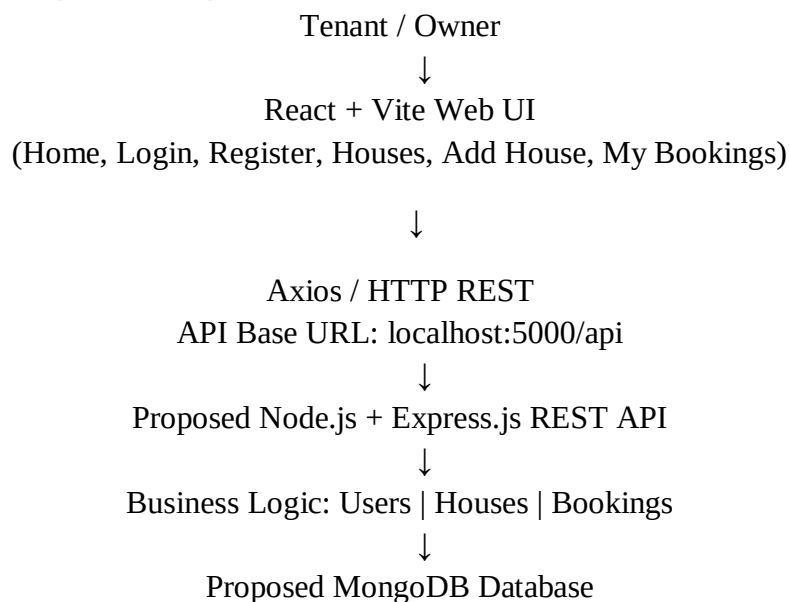
7.2 Non-functional Requirements

NFR No.	Requirement	Description
NFR-1	Usability	Interface should be simple, responsive, and easy to navigate.
NFR-2	Security	Passwords should be hashed on the server; protected operations should validate JWT and role server-side.

NFR-3	Reliability	API errors should be handled without crashing the UI; data operations should return clear status responses.
NFR-4	Performance	House listings and filters should respond quickly; database queries should be indexed where appropriate.
NFR-5	Availability	Production deployment should use reliable hosting, health checks, backups, and monitoring.
NFR-6	Scalability	Frontend and REST backend should be independently deployable and capable of horizontal scaling.
NFR-7	Maintainability	Use modular components, routes, services, controllers, and models.
NFR-8	Compatibility	Support modern desktop and mobile browsers.

8. Technical Architecture and Technology Stack

8.1 Architecture Diagram (Logical)



8.2 Table-1: Components & Technologies

S.No	Component	Description	Technology
1	User Interface	Web interface for tenants and owners	React.js, HTML, CSS
2	Frontend Build Tool	Development and production build	Vite
3	Routing	Client-side navigation and protected routes	react-router-dom
4	API Communication	HTTP requests to REST API	Axios
5	Application Logic – User	Registration, login, session and role handling	React + proposed Node.js/Express
6	Application Logic – House	List, add, update, delete and filter properties	React + proposed Node.js/Express
7	Application Logic – Booking	Create and retrieve bookings	React + proposed Node.js/Express
8	Database	Persistent users, houses and bookings	MongoDB (proposed/assumed)
9	Authentication	Token-based authentication	JWT (expected from frontend token flow)
10	File/Image Handling	Current implementation stores image URL	External image URL; cloud storage can be added
11	Infrastructure	Local development / future cloud deployment	Vite dev server + API server; cloud optional

8.3 Table-2: Application Characteristics

S.No	Characteristic	Description	Technology / Design
1	Open-Source Frameworks	Frontend and proposed backend ecosystem	React, Vite, Express, MongoDB
2	Security Implementations	Authentication, authorization, password protection	JWT, bcrypt, CORS, validation, HTTPS proposed
3	Scalable Architecture	Separation of UI, API, and database layers	3-tier architecture
4	Availability	Can use managed hosting and database redundancy	Cloud deployment, health checks, backups
5	Performance	Client-side filtering; API/database optimization	React state, indexing, pagination/caching proposed

8.4 Important Security Observation

The current frontend checks the owner role before displaying Update Price and Delete buttons, and ProtectedRoute checks for a token before rendering selected pages. These are user-interface controls only. The backend must independently verify the authentication token and authorize the user's role/ownership for every protected API operation.

9. Data Flow and User Stories

9.1 DFD Level 0

External Entities: Tenant and Property Owner. Main System: RentHub. Data Store: Users, Houses, and Bookings. Tenants send registration/login data, search/filter requests, and booking requests. Owners send registration/login data and property-management requests. RentHub returns authentication results, house information, and booking information.

9.2 DFD Level 1

Process	Input	Processing	Output / Data Store
1.0 User Management	Registration/Login details	Validate user and authenticate	Users collection; token/user response
2.0 House Management	House details / update / delete request	Create, read, update, delete house	Houses collection
3.0 House Discovery	Search/filter criteria	Retrieve houses and filter display	House listing
4.0 Booking Management	User ID + House ID	Create/retrieve booking	Bookings collection and booking status

9.3 User Stories

ID	As a...	I want to...	So that...	Acceptance Criteria
US-01	New user	register as tenant or owner	I can use the platform	Valid form submits to /users/register.
US-02	Registered user	log in	I can access authenticated features	Successful login stores token and user.
US-03	Tenant	browse houses	I can find rental options	Houses are fetched and displayed.
US-04	Tenant	search by location	I can narrow results	Listings update according to location text.

US-05	Tenant	filter by type and price	I can find suitable houses	Only matching houses are displayed.
US-06	Tenant	book a house	I can express rental interest	Booking request contains user and house IDs.
US-07	Tenant	view my bookings	I can track booking information	User-specific bookings and status are displayed.
US-08	Owner	add a house	I can advertise a property	House form submits required property information.
US-09	Owner	update a house price	I can keep pricing current	PUT request sends numeric new price.
US-10	Owner	delete a house	I can remove a listing	DELETE request removes the selected listing.

10. Project Planning and Product Backlog

10.1 Suggested Sprint Plan

Sprint	Goal	Major Deliverables
Sprint 1	Project setup and authentication	Vite setup, routing, navbar, registration, login, local session handling
Sprint 2	House discovery	House API integration, cards, location search, type and price filters
Sprint 3	Owner functionality	Add House, update price, delete house, role-based UI
Sprint 4	Booking functionality	Create booking, My Bookings, booking status display
Sprint 5	Quality and release	Responsive UI, validation, security review, testing, documentation

10.2 Product Backlog

Backlog ID	Item	Priority	Story Points	Status
PB-01	Register user	P0	3	Implemented in frontend
PB-02	Login user	P0	3	Implemented in frontend
PB-03	Protected routes	P0	2	Implemented in frontend
PB-04	View houses	P0	5	Implemented in frontend

PB-05	Search and filters	P0	5	Implemented in frontend
PB-06	Add house	P0	5	Implemented in frontend
PB-07	Book house	P0	5	Implemented in frontend
PB-08	View my bookings	P0	5	Implemented in frontend
PB-09	Update house price	P1	3	Implemented in frontend
PB-10	Delete house	P1	3	Implemented in frontend
PB-11	Server-side role authorization	P0	5	Must be verified in backend
PB-12	Image upload	P2	5	Future
PB-13	Payment integration	P2	8	Future

11. User Acceptance Testing (UAT)

TC ID	Scenario	Steps / Input	Expected Result	Status*
TC-01	Register tenant	Enter valid user data and Tenant role	Account created; redirect to login	To Test
TC-02	Register owner	Enter valid user data and Owner role	Owner account created	To Test
TC-03	Login	Enter valid email/password	Token and user stored; redirect home	To Test
TC-04	Invalid login	Enter invalid credentials	Meaningful error message shown	To Test
TC-05	Protected Add House	Open /add-house without token	Redirect to login	To Test
TC-06	Search by location	Enter location text	Only matching houses displayed	To Test
TC-07	Filter by type	Select property type	Only selected type displayed	To Test
TC-08	Filter by price	Select maximum price	Only houses within price displayed	To Test
TC-09	Create booking	Login and click Book Now	Booking created successfully	To Test

TC-10	View bookings	Open My Bookings while logged in	User's bookings displayed	To Test
TC-11	Add house	Submit valid property form	House created and houses page opened	To Test
TC-12	Owner update price	Enter new valid price	Price updated and list refreshed	To Test
TC-13	Owner delete house	Click Delete	House removed and list refreshed	To Test
TC-14	Logout	Click Logout	Token/user removed; redirect login	To Test

*Status is marked 'To Test' because source-code review alone does not prove runtime test execution.

12. Full Stack Development Documentation

12.1 Frontend Architecture

The frontend is a React single-page application. App.jsx defines the application routes. Navbar provides navigation and changes options based on the presence of an authentication token. ProtectedRoute redirects unauthenticated users to the login page. Page components implement the major user workflows, while a centralized Axios instance provides the API base URL.

12.2 Frontend Folder Structure

client/

```

├── src/
│   ├── components/
│   │   ├── Navbar.jsx
│   │   └── ProtectedRoute.jsx
│   ├── pages/
│   │   ├── Home.jsx
│   │   ├── Login.jsx
│   │   ├── Register.jsx
│   │   ├── Houses.jsx
│   │   ├── AddHouse.jsx
│   │   └── MyBookings.jsx
│   ├── services/
│   │   └── api.js
│   ├── App.jsx
│   ├── App.css
│   ├── index.css
│   └── main.jsx
└── index.html

```

└─ package.json
└─ vite.config.js

12.3 Setup Instructions

- Install Node.js compatible with the project's Vite version.
- Open a terminal in the client project folder.
- Run: npm install
- Start the frontend: npm run dev
- Ensure the backend API is running at <http://localhost:5000/api>, or update the Axios baseURL.
- Open the local URL displayed by Vite in the browser.

12.4 Frontend Dependencies

Package	Purpose
React	Component-based user interface
react-dom	Renders React application in the browser
react-router-dom	Client-side routing and navigation
Axios	REST API communication
Vite	Development server and build tool
@vitejs/plugin-react	React integration for Vite
Oxlint	Linting

12.5 API Documentation Based on Frontend Usage

Method	Endpoint	Purpose	Frontend Request Data
POST	/users/register	Register user	name, email, password, role
POST	/users/login	Authenticate user	email, password
GET	/houses	Get all houses	None
POST	/houses/add	Add a house	title, description, location, price, houseType, bedrooms, bathrooms, image, amenities[]
PUT	/houses/:houseId	Update house price	price
DELETE	/houses/:houseId	Delete house	Path parameter: houseId
POST	/bookings/create	Create booking	user, house
GET	/bookings?userId=:userId	Get user's bookings	Query parameter: userId

12.6 Authentication and Authorization

After successful login, the frontend stores `response.data.token` and `response.data.user` in `localStorage`. `ProtectedRoute` checks whether a token exists. The `Navbar` uses the same token to display authenticated navigation. `Houses.jsx` checks `user.role === 'owner'` before displaying update and delete controls. For production security, the backend must validate the token and enforce authorization for protected operations; client-side checks alone are not sufficient.

12.7 Database Schema (Proposed from Frontend Data Model)

Collection	Key Fields
Users	<code>_id</code> , <code>name</code> , <code>email</code> , <code>passwordHash</code> , <code>role</code> (tenant/owner), <code>createdAt</code>
Houses	<code>_id</code> , <code>title</code> , <code>description</code> , <code>location</code> , <code>price</code> , <code>houseType</code> , <code>bedrooms</code> , <code>bathrooms</code> , <code>image</code> , <code>amenities</code> , <code>owner</code> , <code>createdAt</code>
Bookings	<code>_id</code> , <code>user</code> , <code>house</code> , <code>status</code> , <code>bookingDate</code>

12.8 Testing Strategy

- Unit testing for utility and validation logic.
- Component testing for forms, filters, protected routes, and conditional owner controls.
- API integration testing for users, houses, and bookings.
- Authorization testing to ensure tenants cannot perform owner-only backend actions.
- Responsive testing on mobile, tablet, and desktop sizes.
- UAT using the test cases in Section 11.

13. Known Issues and Future Enhancements

13.1 Known / Review Findings

- The submitted source contains frontend code but not the backend implementation, so backend security and database behavior cannot be verified from the supplied files.
- The `axios` base URL is hard-coded to `localhost`, which should be moved to an environment variable for deployment.
- The token is stored in `localStorage`; a production system should carefully address XSS risk and consider secure cookie-based authentication where appropriate.
- Client-side `ProtectedRoute` and owner-role checks do not replace backend authorization.
- The house-type option uses the value `'Apartment'`, which may cause filtering inconsistencies if backend data uses `'Apartment'`.
- Add House is protected by login but the frontend route itself does not restrict the page specifically to owners.
- House image entry currently uses an image URL rather than a managed file-upload workflow.
- Some CSS rules are duplicated and the default Vite index styles may conflict with application layout.

13.2 Future Enhancements

- Email or OTP account confirmation.
- Forgot-password and password-reset workflow.
- Server-side JWT authentication and role/ownership authorization.
- Owner dashboard showing only the owner's properties and booking requests.
- Approve/reject/cancel booking workflow.
- Image upload to cloud object storage.
- Google Maps or other map integration for property location.
- Property availability dates and visit scheduling.
- Online payments and rent receipts.
- Ratings, reviews, favorites, and saved searches.
- Pagination and server-side search for large datasets.
- Notifications by email/SMS/push.
- Admin dashboard for moderation and analytics.
- Deployment with environment-based configuration, monitoring, backups, and CI/CD.

Conclusion

RentHub provides a clear foundation for a full-stack rental-house platform. The submitted frontend already demonstrates the primary user journey: registration, login, house discovery, filtering, property creation, booking, booking history, and owner-facing listing controls. Completing and verifying the backend, strengthening authorization, and adding production deployment practices would turn the current project into a more complete and secure MERN application.